



香港中文大學 (深圳)
The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice

Lecture 8: Dynamic Programming 1

Jingbang Chen
School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen



Borrowing Classroom

- ▶ A school has classroom availability planned for the next n days. On day i , there are r_i classrooms available for borrowing.
- ▶ There are m borrowing requests. The j -th request is described by three positive integers d_j, s_j, t_j , meaning that the applicant wants to borrow d_j classrooms **per day** from day s_j to day t_j inclusive.
- ▶ Applicants do not care about the specific classrooms, locations, or whether the same classrooms are assigned every day. It is sufficient to provide d_j classrooms on each day in the requested interval.
- ▶ Requests are processed in **first-come, first-served** order, i.e., in the order they are given in the input. If a request cannot be fully satisfied, allocation stops immediately, and that applicant must be notified to modify the request.
- ▶ Your task is to determine:
 - ▶ whether all requests can be satisfied;
 - ▶ if not, the index of the first request that cannot be satisfied.



Solution

- ▶ Key 1: Monotonicity

Let $\text{check}(k)$ denote “whether the first k requests can be satisfied.”, apparently it can be binary-searched.

- ▶ Key 2: Fast Feasibility Check

Each request (d, s, t) adds a demand of d classrooms to every day in the interval $[s, t]$. We can do it either with a segment tree, or a different array.



OK, time for dynamic programming!

- ▶ Some very unofficial definition:
- ▶ “You come up with a $f[][]$ representing..., then you figure out a formula.”
- ▶ “You need to make sure the solution can be formed by these $f[][]$.”
- ▶ Yeah...I don't want to repeat the formal definition but it is all about avoiding repeated calculation.



Edit Distance

- ▶ Given two strings word1 and word2, return the minimum number of operations required to convert word1 into word2. You are allowed to perform the following three operations on a string:
- ▶ Insert a character
- ▶ Delete a character
- ▶ Replace a character



Solution

- ▶ $F[i][j]$: the minimum number of operations needed to transform the first i characters of word1 into the first j characters of word2.
- ▶ For each state (i, j) , if the last characters are equal, the answer is exactly the same as for $(i-1, j-1)$;
- ▶ Otherwise, the last step must be one of:
 - Deleting the last character of word1,
 - Inserting the last character of word2,
 - Replacing the last character of word1.



Bursting Ballons

- ▶ You are given n balloons, indexed from 0 to $n - 1$. Each balloon has a number written on it, represented by the array `nums`. You are asked to burst all the balloons. If you burst balloon i , you gain: $\text{nums}[\text{left}] * \text{nums}[i] * \text{nums}[\text{right}]$ coins, where `left` and `right` are the adjacent unburst balloons immediately to the left and right of i .
- ▶ After balloon i is burst, it disappears, so the adjacency changes. Assume there is a virtual balloon with value 1 at both ends of the array: $\text{nums}[-1] = 1$ $\text{nums}[n] = 1$. These virtual balloons cannot be burst.
- ▶ Return the maximum number of coins you can collect by bursting all the balloons.



Solution

- ▶ The hard part is that when you burst a balloon, its neighbors change dynamically, so it is difficult to decide which balloon to burst first.
- ▶ Instead of thinking about: which balloon to burst first,
- ▶ we think about: which balloon is the last one to burst in an interval.



Solution

- ▶ $F[l][r]$: the maximum coins obtainable by bursting all balloons in the interval (l, r) .
- ▶ Transition: Suppose balloon k is the **last** balloon to burst in interval (l, r) . Then at the moment we burst k , every other balloon inside (l, r) has already disappeared, so its neighbors must be exactly l and r .



Strange Printer

- ▶ There is a strange printer with the following two special properties:
- ▶ In one turn, the printer can print a sequence of the same character.
- ▶ At each turn, the printer can print new characters starting from any position and ending at any position, and it will overwrite existing characters.
- ▶ Given a string s , return the minimum number of turns the printer needs to print it.



Solution

- ▶ $F[l][r]$: the minimum number of turns needed to print the substring
- ▶ $dp[l][l] = 1$
- ▶ $dp[l][r] = dp[l + 1][r] + 1$
- ▶ $dp[l][r] = \min(dp[l][r], dp[l + 1][k - 1] + dp[k][r])$
($l < k \leq r$ and $s[k] == s[l]$)



Minimum Cost to Merge Stones

- ▶ You are given an array `stones`, where `stones[i]` is the number of stones in the i -th pile.
- ▶ In one move, you can merge exactly K consecutive piles into one pile.
The cost of this move is equal to the total number of stones in those K piles.
- ▶ Return the minimum cost to merge all piles into one pile.



Solution

- ▶ **Key Observation: Feasibility**
- ▶ Each merge operation takes exactly K piles and turns them into 1 pile, so the total number of piles decreases by: $K - 1$
- ▶ If we start with n piles and want to end with 1 pile, then we must have: $(n - 1) \% (K - 1) == 0$
- ▶ Otherwise, it is impossible, and the answer is -1.



Solution

- ▶ $F[i][j]$: The minimum cost to merge the interval stones $[i..j]$ into the **minimum possible number of piles** allowed by the rules.
- ▶ $dp[i][j] = \min(dp[i][mid] + dp[mid+1][j])$ for all valid mid. (mid = $i, i + (K - 1), i + 2(K - 1), \dots$)
- ▶ If $(j - i) \% (K - 1) == 0$, $dp[i][j] += \text{sum}(i, j)$ (merge into one)



Cherry Pickup

- ▶ You are given an $n \times n$ grid representing a field of cherries, where:
 - 0 means the cell is empty,
 - 1 means the cell contains a cherry,
 - -1 means the cell is blocked by a thorn and cannot be passed through.
- ▶ You start at the top-left corner $(0, 0)$ and want to reach the bottom-right corner $(n - 1, n - 1)$ by moving only **right** or **down**.
- ▶ After reaching the bottom-right corner, you must return to the top-left corner by moving only **left** or **up**.
- ▶ Whenever you pass through a cell containing a cherry, you pick it up, and the cell becomes empty.
- ▶ Return the maximum number of cherries you can collect.



Solution

- ▶ A direct “go there and come back” DP is difficult, because the cherries collected on the first trip affect the second trip.
- ▶ The standard trick is to **transform the two trips into two people walking from $(0,0)$ to $(n-1,n-1)$ at the same time.**



Solution

- ▶ Suppose both walkers have taken exactly k steps.
- ▶ Then if walker 1 is at $(r1, c1)$ and walker 2 is at $(r2, c2)$, we have:
 - $r1 + c1 = k$
 - $r2 + c2 = k$
- ▶ So once we know k , $r1$, and $r2$, the columns are determined:
 - $c1 = k - r1$
 - $c2 = k - r2$
- ▶ $F[k][r1][r2]$: the maximum cherries collected after both walkers have taken k steps, with walker 1 at row $r1$ and walker 2 at row $r2$.
- ▶ $dp[k][r1][r2] =$
max of previous 4 states + cherries collected at current two cells



Remove Boxes

- ▶ You are given several boxes arranged in a row. Each box has a color represented by a positive integer in the array `boxes`.
- ▶ You may repeatedly perform the following operation:
- ▶ choose a contiguous group of k boxes with the same color, remove them, gain $k * k$ points.
- ▶ After removing those boxes, the remaining boxes close up and become adjacent again.
- ▶ Return the maximum points you can obtain by removing all the boxes.



Why This Problem Is Hard

- ▶ A straightforward interval DP like $f[l][r]$ is not enough.
- ▶ The reason is that the best way to remove boxes inside an interval may depend on whether there are **extra boxes of the same color outside the interval** that can later be merged with one endpoint.
- ▶ For example, sometimes it is better **not** to remove a box immediately, because after removing the middle part it can join with another box of the same color and produce a larger square reward.
- ▶ So the state must remember some information from outside the interval.



Solution

- ▶ $F[l][r][k]$ denote the maximum points obtainable from the subarray $\text{boxes}[l..r]$, assuming there are already k extra boxes equal to $\text{boxes}[r]$ attached to the right of r .
- ▶ In other words:
- ▶ we are solving interval $[l, r]$,
- ▶ but box $\text{boxes}[r]$ can be grouped together with k same-colored boxes that have been "saved" from outside.
- ▶ This extra parameter is the key to restoring optimal substructure.



Why This State Works

- ▶ Suppose `boxes[r]` has color c .
- ▶ If there are k extra boxes of color c attached to it, then if we remove `boxes[r]` together with them, the reward is: $(k + 1)^2$
- ▶ if `boxes[r]` stands alone inside the interval,
- ▶ or even larger if we first merge it with matching boxes inside $[l, r-1]$.
- ▶ So k tells us how valuable it is to keep `boxes[r]` for later merging.



Transition

- ▶ **Option 1: Remove boxes[r] now**
- ▶ First solve $[l, r-1]$, then remove boxes[r] together with the k extra boxes:
 - $dp[l][r][k] = dp[l][r-1][0] + (k+1)^2$
- ▶ This corresponds to treating boxes[r] as the last group removed.
- ▶ **Option 2: Merge boxes[r] with some earlier box of the same color**
- ▶ If there exists some m with: $l \leq m < r$ and $boxes[m] == boxes[r]$
- ▶ then instead of removing boxes[r] separately, we may first remove all boxes in (m, r) , so that boxes[m] becomes adjacent to boxes[r].
- ▶ Then boxes[r] can be merged into the group containing boxes[m].
- ▶ So we try: $dp[l][r][k] = \max(dp[l][m][k+1] + dp[m+1][r-1][0])$
- ▶ for every m such that $boxes[m] == boxes[r]$.



Transition

- ▶ $dp[m+1][r-1][0]$: clear the middle part so $boxes[m]$ and $boxes[r]$ can touch;
- ▶ $dp[l][m][k+1]$: now $boxes[m]$ has one more same-colored box attached to it, namely $boxes[r]$, plus the original k extras.
- ▶ Let's try implementing with memorization search!



Summary - Range/Multi-dimensional DP

- ▶ Most important: What should be recorded in the state.
- ▶ Careful about the order to enumerate, and the condition to do transition.
- ▶ Million-dollar advice: Using memoization search!